

SIGSPATIAL '26 Presentation Evaluation & Speaker Script

Paper: *Lexicographic Multi-Preference Routing for Priority-Aware Urban Navigation*

Method: Lexicographic FreqA*

Venue: ACM SIGSPATIAL '26 - Applications Track

Presentation Evaluation

Overall assessment: 9/10

The presentation has a strong conference narrative and a consistent visual style. The technical story now progresses naturally from motivation to data, scoring, route selection, tolerance, algorithm, and evaluation:

Problem -> Insight -> Obstacles -> System -> Preference fields -> Factors -> Route scoring -> ε -> Lexicographic FreqA* -> Algorithm -> Results -> Limitations -> Takeaway.

Final checks before presenting

1. **Remove the duplicate “Preferences on a Common Scale” slide.** In the exported PDF, two consecutive pages contain the same slide; keep the version with the magnified route callouts.
 2. **Treat the pseudocode as optional.** The full algorithm image is too small to read from the audience. If time is tight, skip this slide. If you keep it, discuss only the three concepts on the right and point out that the ε -lexicographic relaxation occurs at line 16.
 3. **Keep the ε qualification precise.** Strict lexicographic optimality is guaranteed at $\varepsilon = 0$. With $\varepsilon > 0$, lower-priority preferences can act inside the near-tie band.
 4. **Use “four data sources” rather than “four open data sources”** if you want to avoid questions about the synthetic historical corpus and the TomTom feed.
 5. **Avoid confusion between the two uses of β .** Earlier, $\beta = 5$ weights fatal crashes; later, β is used as a distance-budget multiplier. Consider calling the second one “budget multiplier” when speaking.
 6. **On the limitations slide,** “the framework applies to preference fields satisfying our scoring assumptions” is more precise than saying it applies to any preference field.
-

Speaker Script

Slide 1 - Title

Target: 30-40 seconds

Good morning everyone. I'm Abdallah Abdelwahab from the University of Alabama, and today I'll present our work, **Lexicographic Multi-Preference Routing for Priority-Aware Urban Navigation**.

The main question behind this work is simple: **why should the navigation system decide what matters most to the user?**

Travel time is important, but in many situations it isn't the only thing we care about. We may care more about safety, congestion, familiar roads, or avoiding signal-heavy corridors.

Our goal is to let the user explicitly decide which of these preferences comes first.

Transition: So let me start with the limitation of current navigation systems.

Slide 2 - The Problem

Target: 45-55 seconds

Most navigation systems optimize mainly for travel time and expose a small number of binary options, such as avoiding tolls, highways, or ferries.

But users often have preferences that don't fit into these switches.

For example, a parent driving at night might prioritize safety. A courier may care about traffic. Another driver may want to avoid routes with many controlled intersections.

More importantly, when these objectives conflict, today's systems don't really let us say **which one matters more**.

That is the problem we're addressing.

Slide 3 - Our Insight

Target: about 40 seconds

Our key insight is that people don't naturally think in numerical weights.

A user usually doesn't say, "give safety a weight of point six and traffic point three."

Instead, people naturally express preferences as an **order**: “Safety first, then the usual route.”

So instead of asking the user for weights, we ask for an ordered preference list.

At strict lexicographic priority, the first preference decides first. Lower preferences are considered only when the higher one cannot clearly distinguish the candidates.

Transition: That sounds simple conceptually, but it introduces two major challenges.

Slide 4 - Two Obstacles

Target: about 50 seconds

The first challenge is data.

Safety, historical popularity, traffic speed, and intersection delay do not naturally exist as comparable edge costs on the road graph. They come from different datasets, in different units and formats.

So first, we need to transform all of them into a common scoring model.

The second challenge is optimization.

A weighted sum collapses the objectives and can allow a lower-priority criterion to compensate for the user’s top preference.

Pareto methods preserve multiple objectives, but typically return several alternatives.

We want something different: **one route that follows the stated priority order, at interactive speed.**

Slide 5 - System Overview

Target: 55-65 seconds

This is the complete system.

On the left, we start from our data sources: OpenStreetMap information, crash records, a historical-route corpus, and traffic information.

Offline, these sources are aligned with the road graph and transformed into preference-specific scores.

We also build a coarse tile graph and precompute heuristic information.

At query time, the user provides a source, a destination, and an ordered preference list - for example safety first, then traffic, then historical popularity.

Live traffic can also be incorporated at query time.

Finally, Lexicographic FreqA* uses those scores and heuristics to return one route.

Transition: The first question, then, is: how do these very different datasets become comparable routing preferences?

Slide 6 - Preferences on a Common Scale

Target: 50-60 seconds

These maps show why the preferences really are different spatial signals.

On the left we have crash exposure, historical usage, and signal density.

The magnified regions make the source, destination, and short route segment easier to see.

Notice that high-density areas occur in different locations for different preferences.

We transform each preference into factors in the range $(0, 1]$, where larger values are more desirable.

So when safety is prioritized, the search is encouraged away from crash-heavy areas. The same principle applies to traffic and signal delay.

Because these fields are spatially different, changing the preference order can genuinely change the route.

Slide 7 - Edge and Node Factors

Target: 40-45 seconds

Each preference can affect an **edge**, a **node**, or both.

Historical popularity is primarily an edge-level factor.

Safety has both an edge factor for mid-block crash exposure and a node factor for intersection crashes.

Traffic also has both: an edge factor representing road speed or congestion and a node factor representing intersection control.

This distinction matters because entering an intersection can impose a penalty independently of the road segment used to reach it.

Slide 8 - Building the Factors

Target: 65-75 seconds

Here I show how the three factors are constructed. I won't go through every term, but I'll highlight the intuition.

For **safety**, more crashes - particularly fatal crashes - reduce the score. Mid-block crashes affect edges, while intersection crashes affect nodes. The outgoing edge values are normalized locally.

For **historical popularity**, we use the fraction of routes toward a destination tile that previously used a particular edge. We use destination tiles rather than exact destination nodes to get more stable statistics.

For **traffic**, the edge score uses the ratio between current speed and free-flow speed. A road moving close to free-flow gets a high value; congestion reduces it. At the node level, signals, stop signs, yields, and crossings impose different penalties.

The important point is that these preference scores remain **separate**. We do not multiply safety, traffic, and history together into one weighted objective.

Transition: Now that we have the individual factors, we can define how an entire route is scored.

Slide 9 - Transition Score -> Route Score -> Best Route

Target: 55-65 seconds

This slide summarizes the mathematical formulation in three steps.

First, every traversal combines an edge factor and the node factor of the intersection we enter.

Second, we multiply those transition scores along the path to obtain the route score for one preference.

Since every factor is between zero and one, higher route scores are better.

Finally, for multiple preferences, we keep the route scores as a vector.

We compare that vector lexicographically: first σ_1 , then σ_2 , and so on.

So the preferences remain independent rather than being collapsed into one number.

Transition: But strict lexicographic comparison creates one practical issue.

Slide 10 - Why Introduce ε ?

Target: 55-65 seconds

Route scores are real-valued products, so exact equality is extremely rare.

Strict lexicographic optimization is perfectly valid, but in practice the first preference can distinguish almost every pair of candidates, meaning the lower preferences rarely get an opportunity to influence the route.

We therefore introduce a relative tolerance, ε .

If two candidate scores on the current preference differ by no more than this relative tolerance, we treat them as a near-tie and move to the next preference.

When $\varepsilon = 0$, we recover strict lexicographic priority.

For a small positive ε , secondary preferences can act only when the primary scores are sufficiently close.

Slide 11 - Lexicographic FreqA*

Target: 65-75 seconds

Our search combines two ideas.

The first is **lexicographic search**. Each search state carries a vector of preference scores rather than one scalar cost.

The second is the **tile-graph heuristic**.

We coarsen the road network into tiles and compute an optimistic estimate of the best remaining score from each tile to the destination.

This gives us an admissible A*-style heuristic, allowing the search to focus on promising regions rather than exploring the full road network.

Importantly, these preference-specific heuristics can be reused for different user orderings.

At $\varepsilon = 0$, the method is provably lexicographically optimal.

Slide 12 - Algorithm

Target: 40-50 seconds; optional if time is tight

I'm not going to walk through Algorithm 2 line by line. There are only three things I want to highlight.

First, the accumulated vector $\vec{g}$ stores the preference scores of the path discovered so far.

Second, the tile heuristic $\vec{h}$ gives an optimistic estimate of the remaining score, and together they form the priority vector $\vec{f}$.

Third - and this is where the previous slide connects to the algorithm - the ε -lexicographic comparison appears in the relaxation step, at **line 16**.

The priority queue itself uses a strict lexicographic ordering; ε is used when deciding whether the newly discovered vector improves the one already stored at a node.

Transition: Now let me show what this actually does to the routes.

Slide 13 - Result 1: The Order Matters

Target: 45-55 seconds

Here we use the same origin and destination but optimize for different preference fields.

The important result is that these are not tiny variations of the same path.

The three offline preference-first routes have pairwise edge overlap of at most **2.6 percent**.

In other words, changing what the user prioritizes can fundamentally reshape the route.

That is exactly the behavior we want from a priority-aware routing system.

Slide 14 - Result 2: Fast and Optimal

Target: 60-70 seconds

The next question is whether this added flexibility makes the search expensive.

For the single-preference FreqA* case, the mean query time is **0.42 milliseconds** on the 256-by-256 grid.

Compared with FreqDijkstra on the same historical-frequency objective, this gives a **97.8-times speedup** and about **80 percent fewer node expansions**, while matching the optimal FreqScore on all 197 queries.

The multi-preference Lex-FreqA* remains very fast as well, at **1.13 milliseconds**.

So introducing preference-aware search does not prevent interactive routing.

Speaker caution: 0.42 ms is FreqA, *not* Lex-FreqA.

Slide 15 - Live Traffic

Target: 45-55 seconds

We also tested whether the same framework can respond to dynamic information.

Here we run the same traffic-first query at two different times using live TomTom traffic.

As congestion changes, the preferred route changes with it.

There is no need to redesign the algorithm - only the traffic factors change.

The routing search itself remains around **2.2 milliseconds**.

This is search time; the external traffic-data acquisition is separate.

Slide 16 - The Cost of Preference

Target: 60-70 seconds

Preference-aware routes are not free.

If we prioritize historical popularity, safety, or traffic instead of pure distance, the route can become longer.

For example, the safety-first route is roughly 60 percent longer than the shortest route in this experiment.

We view that not as algorithmic inefficiency, but as the measurable cost of satisfying a different user objective.

We therefore also introduce an optional distance budget.

As the budget becomes tighter, the detour decreases, but we give up some safety quality.

For example, with a budget multiplier of 2, the detour drops to about **40.5 percent** while retaining about **85.3 percent** of the unconstrained safety score.

So the system exposes an interpretable tradeoff when the user wants one.

Speaker caution: If the slide still uses β for the distance budget, clarify that this is the budget multiplier, not the fatal-crash weight from the earlier slide.

Slide 17 - Limitations

Target: 45-55 seconds

There are three important limitations.

First, the historical corpus in this experiment is synthetic, so we are not claiming that the historical preference reproduces real driver behavior.

Second, our safety score measures recorded crash exposure, not per-vehicle crash risk, because the crash counts are not normalized by traffic volume.

Third, the evaluation is currently limited to Manhattan.

These limitations affect the interpretation of the preference fields, but the search framework itself can accept other fields that satisfy the same scoring assumptions.

Extending this to real trajectory data and additional cities is a natural next step.

Slide 18 - Takeaway

Target: 35-45 seconds

So the main takeaway is simple.

A user should be able to tell the navigation system:

“Safest first, then fastest.”

And the system should return one route that explicitly respects that order, rather than hiding the tradeoff inside a weighted sum or asking the user to choose from a Pareto set.

Lexicographic FreqA* provides that framework, with strict lexicographic optimality at $\varepsilon = 0$, efficient tile-based search, and support for live traffic.

Ultimately, we want the user - not the routing system - to decide what matters first.

Thank you. I'm happy to take questions.

Three Sentences to Memorize

Opening

The main question behind this work is simple: why should the navigation system decide what matters most to the user?

Core contribution

Instead of collapsing preferences into weights, we keep them separate and compare them according to the user's priority order.

Closing

Ultimately, we want the user - not the routing system - to decide what matters first.

Timing Note

The full script is designed for roughly **13-15 minutes** when delivered conversationally. If you need to shorten it, the easiest slide to skip is the pseudocode slide; you can also compress Slides 7-9 by explaining only the intuition behind the equations rather than reading the notation.